

Migration: Porting a Game from PC to Handheld

Code Samples

```
// Often used fixed numbers
#define ONE_FX 0x00010000
#define TWO_FX 0x00020000

// fixed conversion routines and absolute value
#define _fixed_shift 16 // our fixed point is 16.16, used throughout
const float FXFACTOR = (1.0f/65536.0f);
const float FXINV = 65536.0f;

__inline GLfixed f2x(float f) { return (GLfixed)((f)*FXINV); }
__inline float x2f(GLfixed x) { return (float)(x)*FXFACTOR; }
__inline GLfixed i2x(int i) { return (i)<<_fixed_shift; }
__inline int x2i(GLfixed x) { return int(x)>>_fixed_shift; }
__inline GLfixed absfx(GLfixed x) { return (x<0) ? -x : x; }

// multiply two fixed values
__inline GLfixed MulFX(GLfixed x, GLfixed y)
{
    __int64 r = ((__int64)x) * y;
    return (GLint)(r >> 16);
}

// divide two fixed values
__inline GLfixed DivFX(const GLfixed x, const GLfixed y)
{
    return (GLint)(((__int64)x)<<16)/(y));
}

// Rewrite of OpenGL ES transform stack to work around the QueryMatrixOES
extern GLfixed modelview[16];
extern GLfixed projection[16];
extern vector<GLfixed *>modelstack;
extern vector<GLfixed *>projstack;

// intended to emulate glLoadIdentity
__inline void identity(GLfixed *curMtrx)
{
    curMtrx[0] = ONE_FX; curMtrx[4] = 0; curMtrx[8] = 0; curMtrx[12] = 0;
    curMtrx[1] = 0; curMtrx[5] = ONE_FX; curMtrx[9] = 0; curMtrx[13] = 0;
    curMtrx[2] = 0; curMtrx[6] = 0; curMtrx[10] = ONE_FX; curMtrx[14] = 0;
    curMtrx[3] = 0; curMtrx[7] = 0; curMtrx[11] = 0; curMtrx[15] = ONE_FX;
    glLoadIdentity();
}
```

```

// push a projection matrix onto projstack
__inline void pushProjMatrix()
{
    GLfixed *toAdd = new GLfixed[16];
    for(int i = 0; i < 16; i++)
        toAdd[i] = projection[i];
    projstack.add(toAdd);
}

// push a modelview matrix onto modelstack
__inline void pushModelMatrix()
{
    GLfixed *toAdd = new GLfixed[16];
    for(int i = 0; i < 16; i++)
        toAdd[i] = modelview[i];
    modelstack.add(toAdd);
}

// pop the top projection matrix from projstack
__inline void popProjMatrix()
{
    // if empty, top is always identity
    if(projstack.length() == 0)
        identity(projection);
    else
    {
        GLfixed *temp = projstack.pop();
        for(int i = 0; i < 16; i++)
            projection[i] = temp[i];
        delete temp;

        // load the popped matrix into OpenGL, assumes state is set correctly
        glLoadMatrixx(projection);
    }
}

// pop the top modelview matrix from modelstack
__inline void popModelMatrix()
{
    // if empty, top is always identity
    if(modelstack.length() == 0)
        identity(modelview);
    else
    {
        GLfixed *temp = modelstack.pop();
        for(int i = 0; i < 16; i++)
            modelview[i] = temp[i];
        delete temp;

        // load the popped matrix into OpenGL, assumes state is set correctly
        glLoadMatrixx(modelview);
    }
}

```

```

// 4x4 fixed point matrix multiply
inline void matrixMultiply(GLfixed *in1, GLfixed *in2, GLfixed *out)
{
    if(!out || !in1 || !in2) return;

    out[0] = MulFX(in1[0],in2[0]) + MulFX(in1[1],in2[4]) + MulFX(in1[2],in2[8]) +
        MulFX(in1[3],in2[12]);
    out[1] = MulFX(in1[0],in2[1]) + MulFX(in1[1],in2[5]) + MulFX(in1[2],in2[9]) +
        MulFX(in1[3],in2[13]);
    out[2] = MulFX(in1[0],in2[2]) + MulFX(in1[1],in2[6]) + MulFX(in1[2],in2[10]) +
        MulFX(in1[3],in2[14]);
    out[3] = MulFX(in1[0],in2[3]) + MulFX(in1[1],in2[7]) + MulFX(in1[2],in2[11]) +
        MulFX(in1[3],in2[15]);
    out[4] = MulFX(in1[4],in2[0]) + MulFX(in1[5],in2[4]) + MulFX(in1[6],in2[8]) +
        MulFX(in1[7],in2[12]);
    out[5] = MulFX(in1[4],in2[1]) + MulFX(in1[5],in2[5]) + MulFX(in1[6],in2[9]) +
        MulFX(in1[7],in2[13]);
    out[6] = MulFX(in1[4],in2[2]) + MulFX(in1[5],in2[6]) + MulFX(in1[6],in2[10]) +
        MulFX(in1[7],in2[14]);
    out[7] = MulFX(in1[4],in2[3]) + MulFX(in1[5],in2[7]) + MulFX(in1[6],in2[11]) +
        MulFX(in1[7],in2[15]);
    out[8] = MulFX(in1[8],in2[0]) + MulFX(in1[9],in2[4]) + MulFX(in1[10],in2[8]) +
        MulFX(in1[11],in2[12]);
    out[9] = MulFX(in1[8],in2[1]) + MulFX(in1[9],in2[5]) + MulFX(in1[10],in2[9]) +
        MulFX(in1[11],in2[13]);
    out[10] = MulFX(in1[8],in2[2]) + MulFX(in1[9],in2[6]) + MulFX(in1[10],in2[10]) +
        MulFX(in1[11],in2[14]);
    out[11] = MulFX(in1[8],in2[3]) + MulFX(in1[9],in2[7]) + MulFX(in1[10],in2[11]) +
        MulFX(in1[11],in2[15]);
    out[12] = MulFX(in1[12],in2[0]) + MulFX(in1[13],in2[4]) + MulFX(in1[14],in2[8]) +
        MulFX(in1[15],in2[12]);
    out[13] = MulFX(in1[12],in2[1]) + MulFX(in1[13],in2[5]) + MulFX(in1[14],in2[9]) +
        MulFX(in1[15],in2[13]);
    out[14] = MulFX(in1[12],in2[2]) + MulFX(in1[13],in2[6]) + MulFX(in1[14],in2[10]) +
        MulFX(in1[15],in2[14]);
    out[15] = MulFX(in1[12],in2[3]) + MulFX(in1[13],in2[7]) + MulFX(in1[14],in2[11]) +
        MulFX(in1[15],in2[15]);
}

// intended to emulate glRotatex
inline void rotate(GLfixed *curMtrx, GLfixed angle, GLfixed x, GLfixed y, GLfixed z)
{
    if(!curMtrx) return;

    GLfixed m[16];

    const GLfixed c = f2x((float)cos(x2f(angle) * (3.14159f/180.0f)));
    const GLfixed s = f2x((float)sin(x2f(angle) * (3.14159f/180.0f)));
    const GLfixed mag = MulFX(x, x) + MulFX(y, y) + MulFX(z, z);

    // inverse sqrt to normalize
    // consider Newton approximation for speed
    float w = 1/sqrt(x2f(mag));
    GLfixed n = f2x(w);

```

```

x = MulFX(x, n);
y = MulFX(y, n);
z = MulFX(z, n);

m[0] = MulFX(x, MulFX(x, ONE_FX - c)) + c;
m[1] = MulFX(x, MulFX(y, ONE_FX-c)) + MulFX(z, s);
m[2] = MulFX(z, MulFX(x, ONE_FX-c)) - MulFX(y, s);
m[3] = 0;

m[4] = MulFX(x, MulFX(y, ONE_FX-c)) - MulFX(z, s);
m[5] = MulFX(y, MulFX(y, ONE_FX - c)) + c;
m[6] = MulFX(y, MulFX(z, ONE_FX-c)) + MulFX(x, s);
m[7] = 0;

m[8] = MulFX(z, MulFX(x, ONE_FX-c)) + MulFX(y, s);
m[9] = MulFX(y, MulFX(z, ONE_FX-c)) - MulFX(x, s);
m[10] = MulFX(z, MulFX(z, ONE_FX - c)) + c;
m[11] = 0;

m[12] = 0;
m[13] = 0;
m[14] = 0;
m[15] = ONE_FX;

GLfixed out[16];

matrixMultiply(m, curMtrx, out);

for(int j = 0; j < 16; j++)
    curMtrx[j] = out[j];

// load newly rotated matrix into OpenGL
glLoadMatrixx(curMtrx);
}

// intended to emulate glTranslatex
inline void translate(GLfixed *curMtrx, const GLfixed x, const GLfixed y, const GLfixed z)
{
    if(!curMtrx) return;

    GLfixed m[16];

    m[0] = ONE_FX;
    m[1] = 0;
    m[2] = 0;
    m[3] = 0;

    m[4] = 0;
    m[5] = ONE_FX;
    m[6] = 0;
    m[7] = 0;

    m[8] = 0;
    m[9] = 0;
    m[10] = ONE_FX;
    m[11] = 0;

```

```

        m[12] = x;
        m[13] = y;
        m[14] = z;
        m[15] = ONE_FX;

        GLfixed out[16];

        matrixMultiply(m, curMtrx, out);

        for(int i = 0; i < 16; i++)
            curMtrx[i] = out[i];

        // load newly translated matrix into OpenGL
        glLoadMatrixx(curMtrx);
    }

// intended to emulate glFrustumx
__inline void frust(GLfixed *curMtrx, const GLfixed left, const GLfixed right,
                    const GLfixed bottom, const GLfixed top, const GLfixed zNear, const GLfixed zFar)
{
    if(!curMtrx) return;

    GLfixed m[16], out[16];
    m[0] = DivFX(MulFX(TWO_FX, zNear), right-left);
    m[1] = 0;
    m[2] = 0;
    m[3] = 0;

    m[4] = 0;
    m[5] = DivFX(MulFX(TWO_FX, zNear), top-bottom);
    m[6] = 0;
    m[7] = 0;

    m[8] = DivFX(right+left, right-left);
    m[9] = DivFX(top+bottom, top-bottom);
    m[10] = DivFX(-(zFar+zNear), zFar-zNear);
    m[11] = -ONE_FX;

    m[12] = 0;
    m[13] = 0;
    m[14] = DivFX(-MulFX(MulFX(TWO_FX, zFar), zNear), zFar-zNear);
    m[15] = 0;

    matrixMultiply(m, curMtrx, out);
    for(int i = 0; i < 16; i++)
        curMtrx[i] = out[i];

    // load new matrix into OpenGL
    glLoadMatrixx(curMtrx);
}

// intended to emulate glOrthox
__inline void ortho(GLfixed *curMtrx, const GLfixed left, const GLfixed right, const GLfixed bottom,
                    const GLfixed top, const GLfixed zNear, const GLfixed zFar)
{
    if(!curMtrx) return;

```

```

    GLfixed m[16], out[16];

    m[0] = DivFX(TWO_FX, right - left);
    m[1] = 0;
    m[2] = 0;
    m[3] = 0;

    m[4] = 0;
    m[5] = DivFX(TWO_FX, top - bottom);
    m[6] = 0;
    m[7] = 0;

    m[8] = 0;
    m[9] = 0;
    m[10] = -DivFX(TWO_FX, zFar - zNear);
    m[11] = 0;

    m[12] = DivFX(-(right+left), right - left);
    m[13] = DivFX(-(top+bottom), top - bottom);
    m[14] = DivFX(-(zFar+zNear), zFar - zNear);
    m[15] = ONE_FX;

    matrixMultiply(m, curMtrx, out);

    for(int i = 0; i < 16; i++)
        curMtrx[i] = out[i];

    // load new matrix into OpenGL
    glLoadMatrixx(curMtrx);
}

// this code is to get the absolute path from the relative for PocketPC
// relativepath represents the path to the file you are looking for based on the current executable
// absfilepath is the output, relativepath concatenated with the executable's absolute path
wchar_t execpath[MAX_PATH];
char absfilepath[MAX_PATH];
int i;

GetModuleFileName(GetModuleHandle(NULL), execpath, MAX_PATH);
for(i = wcslen(execpath); i && execpath[i]!='\\'; i--);
execpath[i+1] = L'0';
WideCharToMultiByte(CP_ACP, 0, execpath, MAX_PATH, absfilepath, MAX_PATH, NULL, NULL);

strcat(absfilepath, relativepath);

/* Generate coordinates of a point on a sphere with player
at the center, based on his orientation. Cast a ray along
this line (the crosshair for FPS games), searching bounding areas of
world objects for any that it crosses. Find the one that
is the closest of those it crosses. */

vector a;

// cast ray
a.x = 1000 * (float)sin(player->pitch) * (float)cos(player->yaw);

```

```

a.y = 1000 * (float)sin(player->pitch) * (float)sin(player->yaw);
a.z = 1000 * (float)cos(player->pitch);

// do collision testing for this vector against objects in world
...
// sort objects that collide in z-order to determine closest
...

// Special helper class for storing rgb values during mipmapping
class Texel
{
public:
    GLubyte r, g, b;

    Texel(GLubyte red, GLubyte green, GLubyte blue) { r = red; g = green; b = blue; }
    Texel() { r = 0; g = 0; b = 0; }
};

// Get an rgb texel from the memory pointer at the offset given,
// wont work for rgba textures unless modified
__inline Texel GetTexel(const GLubyte *s, int offset) { return Texel(s[offset], s[offset+1], s[offset+2]); };

// Average of four mipmap algorithm for square textures
void makeMipMaps(Surface *parentTexture)
{
    // drop out if not a square texture
    if(parentTexture->w != parentTexture->h)
    {
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexEnvx(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, parentTexture->w, parentTexture->h,
                    0, GL_RGB, GL_UNSIGNED_BYTE, parentTexture->pixels);
        return;
    }
    else
    {
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
                        GL_LINEAR_MIPMAP_LINEAR); // mips on
        glTexEnvx(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE);
    }

    // algorithm adapted from Vincent 3D Rendering Library
    // original code by Hans-Martin Will

    unsigned int width = parentTexture->w;
    unsigned int height = parentTexture->h;
    int level = 0;

```

```

// destroy original to save memory
GLubyte *texture = (GLubyte *)parentTexture->pixels;

// 0th level is the actual texture as given
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB,
             GL_UNSIGNED_BYTE, texture);

while (width > 1 || height > 1)
{
    width = width >> 1;
    height = height >> 1;

    ++level;

    for(unsigned int row = 0; row < height; ++row)
    {
        for(unsigned int col = 0; col < width; ++col)
        {
            Texel p00 = GetTexel(texture, (2 * row * (width * 2) + 2 * col) * 3);
            Texel p01 = GetTexel(texture, (2 * row * (width * 2) + 2 * col + 1) *
                                   3);
            Texel p10 = GetTexel(texture, ((2 * row + 1) * (width * 2) + 2 * col) *
                                   3);
            Texel p11 = GetTexel(texture, ((2 * row + 1) * (width * 2) + 2 * col +
                                   1) * 3);

            Texel result =
                Texel((p00.r + p01.r + p10.r + p11.r) / 4,
                     (p00.g + p01.g + p10.g + p11.g) / 4,
                     (p00.b + p01.b + p10.b + p11.b) / 4);

            texture[(row * width + col)*3] = result.r;
            texture[(row * width + col)*3+1] = result.g;
            texture[(row * width + col)*3+2] = result.b;

        }
    }
    glTexImage2D(GL_TEXTURE_2D, level, GL_RGB, width, height, 0, GL_RGB,
                 GL_UNSIGNED_BYTE, texture);
}
}

```